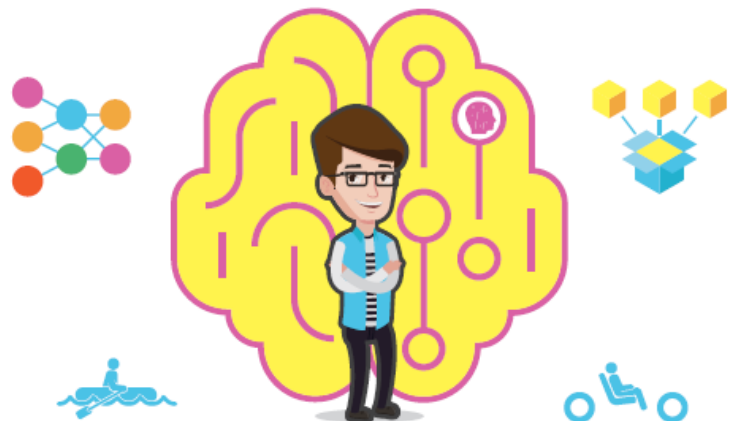


실습하며 배우는
딥러닝 입문
with **Kaggle**



타이타닉 생존자 예측부터 자율주행 자동차까지

[보조자료] 제공(www.booker.co.kr)

장원두 지음

· 파이썬 소스 코드



실습하며 배우는

딥러닝 입문 WITH KAGGLE

3장: 파이썬과 코딩의 기초 (2부)

목 차

3.6 조건문

3.7 고급 데이터 타입

3.8 반복문

3.9 Numpy 배열

3.10 사용자 정의 함수



조건문

- 상황에 따라 서로 다른 코드를 실행
 - if, else, 콜론 (:) 등으로 분기

if 조건식:

조건식이 참인 경우 수행할 명령 1

조건식이 참인 경우 수행할 명령 2

...

else:

조건이 참이 아닌 경우 수행할 명령 1

조건이 참이 아닌 경우 수행할 명령 2

...

- 들여쓰기 주의 필요

- 조건식에 사용하는 비교 기호

기호	사용예	의미
==	a==b	a와 b 가 같다
!=	a != b	a와 b 가 다르다
<	a<b	a가 b보다 작다
>	a>b	a가 b보다 크다
<=	a<=b	a가 b보다 작거나 같다
>=	a>=b	a가 b보다 크거나 같다



조건문

- Sin함수의 결과에 따라 다른 코드가 실행되는 프로그램

```
import numpy as np
a = np.sin(np.pi/3)
print('결과는', a, '입니다')
```

a>0인 경우

그 외의 경우

```
print('사인값은 양수입니다')
```

```
print('사인값은 양수가 아닙니다')
```

실습



```
import numpy as np
a = np.sin(np.pi/3)
print('결과는', a, '입니다')
if a>0:
    print('사인값은 양수입니다')
else:
    print('사인값은 양수가 아닙니다')
```

- 들여쓰기 주의 필요



다중 조건문

- 여러 개의 분기를 가지는 프로그램
- If, elif, else로 경우의 수를 나눔

if 조건식 1:

if 조건식 1-1:

조건식1과 조건식 1-2가 모두 참인 경우 수행할 명령

elif 조건식 1-2:

조건식1은 참, 조건식 1-1은 거짓, 1-2는 참인 경우 수행할 명령

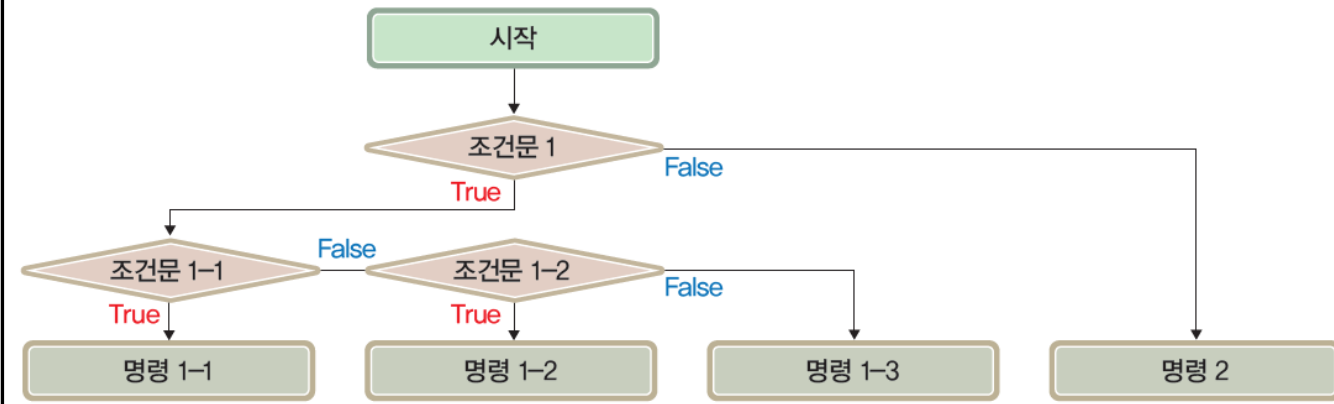
else:

조건식1은 참이고, 조건식 1-1과 1-2는 거짓인 경우 수행할 명령

else 조건식 2:

조건식2가 참인 경우 수행할 명령 1

조건식2가 참인 경우 수행할 명령 2



[그림 3-19] 다중 조건문의 플로우 차트



조건식의 조합

- 두 개 이상의 조건이 동시에 만족하는 경우
- 여러 조건 중 하나 이상의 조건이 만족하는 경우 등을 상
- and, or 키워드, 소괄호를 사용하여 조합

조건식1 and 조건식2 and 조건식3 ...

조건식1 or 조건식2 or 조건식3 ...

실습

- 2차원 좌표가 위치하는 사분면을 계산하는 코드

```
▶ place = 0
if x > 0 and y > 0:
    place = 1
elif x < 0 and y > 0:
    place = 2
elif x < 0 and y < 0:
    place = 3
elif x > 0 and y < 0:
    place = 4
if place==0:
    print('점이 축 위에 있습니다.')
else:
    print(f'점은 {place}사분면에 있습니다.')
```

- 조건문의 조합 없이 작성한 코드

```
▶ place = 0
if y > 0:
    if x > 0:
        place = 1
    elif x < 0:
        place = 2
elif y < 0:
    if x < 0:
        place = 3
    elif x > 0:
        place = 4
if place==0:
    print('점이 축 위에 있습니다.')
else:
    print(f'점은 {place}사분면에 있습니다.')
```



조건식의 조합 실습 (or)

- x, y 의 값 중 하나라도 100보다 크면 '크다', 둘 모두 100보다 작거나 같으면 '작다'라고 화면에 출력하는 코드
- 두 조건 $x > 100, y > 100$ 중 하나만 만족하면 되므로, 키워드 `or`을 사용하여 두 조건식을 연결함



```
if x > 100 or y > 100:  
    print('크다')  
else:  
    print('작다')
```




조건문의 사용 실습 (3.6.4)

1) 변수 x 에는 다음과 같이 -10 에서 10 사이의 정수가 저장되어 있을 때,

$x = -5$

이 값이 양수이면 2를 곱하고, 양수가 아니면 2로 나누어 다시 x 에 저장하는 코드를 작성하라.



```
if x > 0:  
    x = x * 2  
else:  
    x = x / 2
```




조건문의 사용 실습 (3.6.4)

2) 변수 v 에 다음과 같이 임의의 실수값이 저장되어 있을 때,

$v = -1.5$

$\sin(v)$ 의 값이 양수이면 PLUS, 음수이면 MINUS, 0이면 ZERO를 출력하는 코드를 작성하라.



```
import numpy as np
sin_v = np.sin(v)

if sin_v > 0:
    print('PLUS')
elif sin_v < 0:
    print('MINUS')
else:
    print('ZERO')
```



조건문의 사용 실습 (3.6.4)

3) 변수 `score`에 다음과 같이 성적(0~100점)이 저장되어 있다고 할 때,

```
score = 85
```

성적이 90점 이상이면 A, A가 아니면서 80점 이상이면 B, 70점 이상이면 C, 60점 이상이면 D, 모두 해당되지 않으면 F를 표시하는 코드를 작성하라.



```
if score >= 90:
    print('A')
elif score >= 80:
    print('B')
elif score >= 70:
    print('C')
elif score >= 60:
    print('D')
else:
    print('F')
```



조건문의 사용 실습 (3.6.4)

4) 변수 `weight` 에는 몸무게, `height`에는 키(미터)가 다음과 같이 저장되어 있을 때,

```
weight = 65.3  
height = 1.78
```

두 변수를 사용하여 `bmi` 지수를 계산하라. `bmi`지수는 몸무게를 키의 제곱으로 나눈 값이다.
`bmi` 지수가 25 이상이면 '비만', 18.5 미만이면 '저체중',
두 경우 모두 해당하지 않으면 '정상'으로 화면에 표시하는 코드를 작성하라.



```
bmi = weight / (height * height) #bmi의 계산  
if bmi >= 25:  
    print('비만')  
elif bmi < 18.5:  
    print('저체중')  
else:  
    print('정상')
```




조건문의 사용 실습 (3.6.4)

5) 변수 `grade`에는 다음과 같이 영어 대문자로 표기된 학점(A ~ F)이 저장되어 있다.

```
grade = 'A'
```

영어로 표기된 학점을 0~100점 사이의 점수로 환산하여 출력하는 코드를 작성하라.
단, A는 95, B는 85, C는 75, D는 65, F는 0점이다.



```
score = 0
if grade=='A':
    score = 95
elif grade=='B':
    score = 85
elif grade=='C':
    score = 75
elif grade=='D':
    score = 65
print(f'The converted score is {score}')
```



목 차

3.6 조건문

3.7 고급 데이터 타입

3.8 반복문

3.9 Numpy 배열

3.10 사용자 정의 함수



데이터의 묶음을 효과적으로 다루기 위한 데이터 타입

리스트

튜플

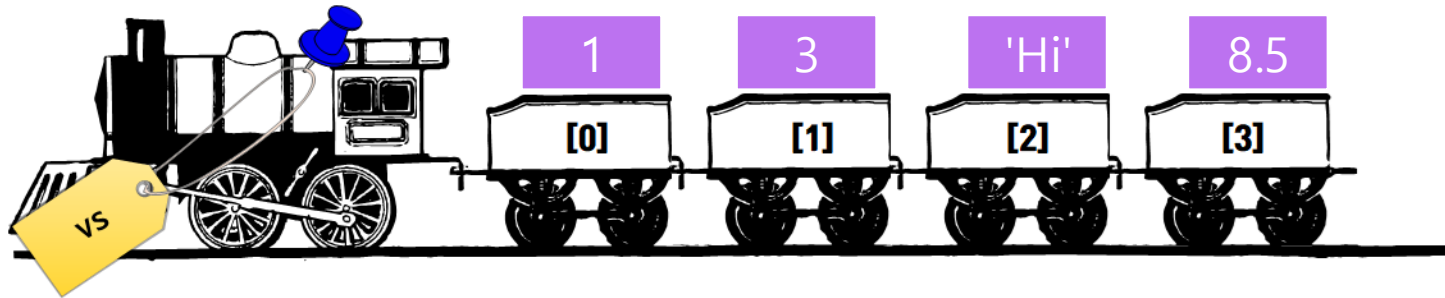
딕셔너리

집합



리스트

- 여러 개의 데이터 공간이 이어져 있는 데이터 타입
- 하나의 이름으로 전체 리스트 관리
- 인덱스: 리스트의 번호. 대괄호를 사용하여 중간에 있는 데이터도 직접적으로 관리(읽기, 수정) 가능



```
vs = [1, 3, 'Hi', 8.5]  
print(vs)  
print(vs[2])
```

리스트

튜플

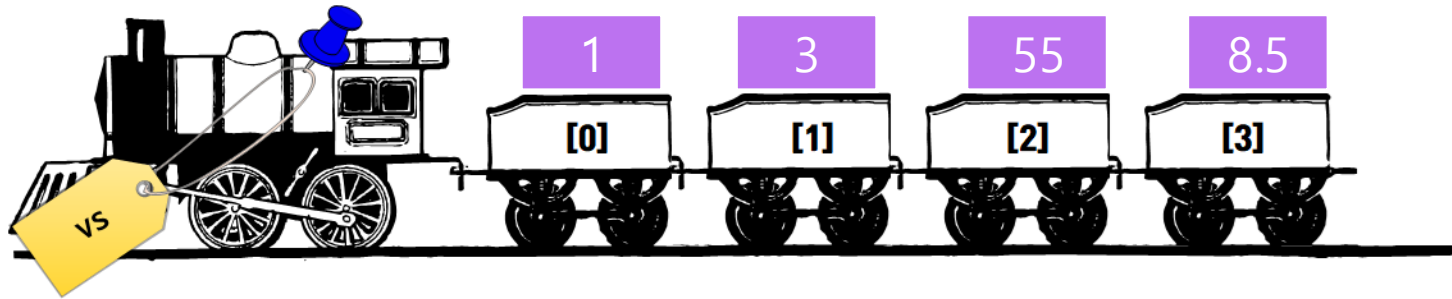
딕셔너리

집합



리스트의 값 바꾸기

- 대괄호를 사용하여 위치를 지정하고 대입 연산자(=)를 사용하여 값 수정



```
vs = [1, 3, 'Hi', 8.5]
print(vs)
print(vs[2])
vs[2] = 55
print(vs[2])
```

리스트

튜플

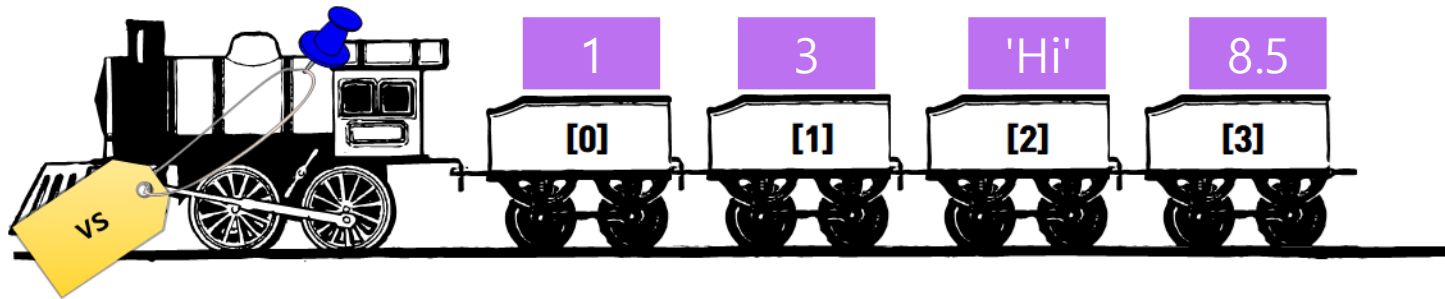
딕셔너리

집합



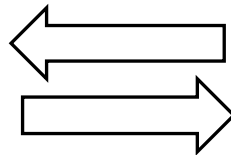
리스트에 데이터 추가

- append 함수를 사용하여 리스트 가장 뒤에 데이터 추가 가능
- 변수에 []를 지정하면 빈 리스트 생성 가능



```
vs = []
vs.append(1)
vs.append(3)
vs.append('Hi')
vs.append(8.5)
```

동일한 결과



```
vs = [1, 3, 'Hi' , 8.5]
```

리스트

튜플

딕셔너리

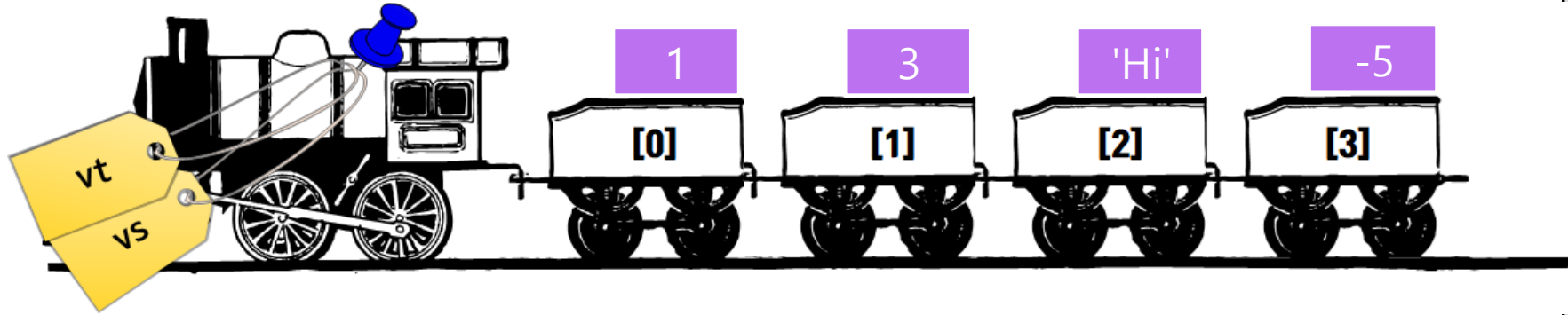
집합



07. 고급 데이터 타입

리스트 객체와 리스트 변수

- 리스트 객체: 메모리 상에서 존재하는 리스트의 실재(實在)
- 리스트 변수: 리스트 객체에 붙어있는 태그



```
vs = [1, 3, 'Hi', 8.5]
vt = vs
vt[3] = -5
print(vs)
```

- vt[3]의 값을 -5로 변경하면, vs[3]의 값 역시 -5로 변화
- 2개의 값이 바뀌는 것이 아니라, 두 변수가 가리키는 객체가 동일하기 때문에 발생하는 현상

리스트

튜플

딕셔너리

집합



07. 고급 데이터 타입

리스트와 range함수

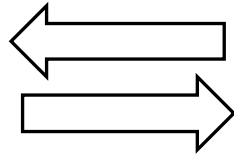
- `range(start, end)`
 - `start ~ end - 1`까지의 연속된 정수를 만든다.
- `range(start, step, end)`
 - `start ~ end - 1`까지의 연속된 정수를 `step` 단위로 만든다.
- `list( range(s,e) )`
 - `s`부터 `e-1`까지의 연속된 정수를 만들고, 이를 리스트 형태로 변환한다
- `a = list(range(s,e))`
 - `s`부터 `e-1`까지의 연속된 정수를 만들고, 이를 리스트 형태로 변환한 후, 변수 `a`가 가리키는 객체에 저장한다.

a에 저장



```
v = list(range(3,9))
```

동일한 결과



```
v = [3,4,5,6,7,8]
```

리스트

튜플

딕셔너리

집합



튜플

- 리스트와 유사
- 차이점
 - 한번 만들면 수정할 수 없음
 - 대괄호 없이 생성



```
tup = 3,4,5  
print(tup[0])
```



```
tup = (3,4,5)  
print(tup[0])
```



```
my_list = [5,8,7]  
tup = tuple( my_list )
```



```
tup[2] = 1
```

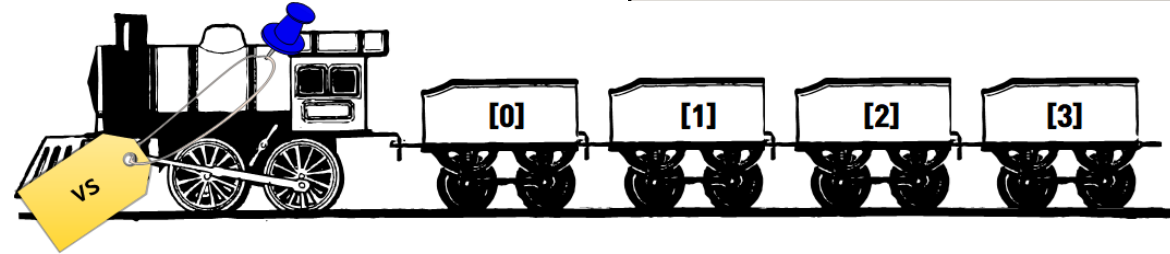


리스트

튜플

딕셔너리

집합





07. 고급 데이터 타입

딕셔너리 (사전)

- 리스트와 유사
- 차이점
 - 항목에 순서가 없음
 - 각 항목에 이름이 붙어 있음



```
d1 = {'Title' : 'The Chronicles of Narnia',
      'Year' : 1950,
      'Author' : 'C. S. Lewis',
      'Country' : 'UK',
      'Genre' : 'Fantasy'}
```

```
print(d1['Title'])
```

```
d1['Author'] = 'Me'
d1['Pages'] = 538
d1.pop('Year')
```

'Title'	'The Chronicles of Narnia'
'Year'	1950
'Author'	'C. S. Lewis'
'Country'	'UK'
'Genre'	'Fantasy'

실습

자신이 좋아하거나 싫어하는 책 2권을 선정하고,
해당 책의 정보를 딕셔너리의 형태로 변수 d2, d3에
저장하라.
딕셔너리에 저장할 항목들의 종류는 위 표와 같이 하라.

리스트

튜플

딕셔너리

집합



딕셔너리 (사전) 실습

딕셔너리 3개로 구성된 리스트를 생성하고 내용을 확인하라

리스트

튜플

딕셔너리

집합



```
d1 = {'Title' : 'The Chronicles of Narnia',  
      'Year' : 1950,  
      'Author' : 'C. S. Lewis',  
      'Country' : 'UK',  
      'Genre' : 'Fantasy'}
```

```
d2 = {'Title' : 'NanJung Ilgi',  
      'Year' : 1598,  
      'Author' : 'Soonshin Lee',  
      'Country' : 'Korea',  
      'Genre' : 'History'}
```



```
d3 = {'Title' : 'Romance of Three Kingdoms',  
      'Year' : 1380,  
      'Author' : 'Kwanjung Na',  
      'Country' : 'China',  
      'Genre' : 'Historical Novel'}
```

```
#딕셔너리의 리스트 생성  
d_list = [d1, d2, d3]
```

```
#내용 확인  
print(d_list[2]['Title'])
```



집합

- 리스트, 딕셔너리와 유사
- 차이점
 - 항목에 순서가 없음
 - 항목에 이름이 없음



```
s = {3, 5, 4}
print(s)
s.add(8)
```

리스트

튜플

딕셔너리

집합



고급 데이터 타입 사용 실습 (3.7.5)

1) 리스트 변수 `v`에 다음과 값이 저장되어 있을 때,

```
v = [3,4,5,6]
```

리스트에 포함된 값들의 평균을 계산하라.



```
avg = ( v[0] + v[1] + v[2] + v[3] ) / 4
```

리스트

튜플

딕셔너리

집합



고급 데이터 타입 사용 실습 (3.7.5)

- 2) 1부터 30사이의 홀수를 순서대로 리스트에 저장하고,
15번째 홀수의 값을 화면에 출력하라.



```
v = list( range(1,30,2) )  
print( v[14] )
```

리스트

튜플

딕셔너리

집합



고급 데이터 타입 사용 실습 (3.7.5)

- 3) 3.7.3장의 실습 코드와 같이 책 세 권의 정보가 리스트형 변수인 `d_list`에 저장되어 있다고 하자. 책 세 권 중 1950년에 출판된 책을 찾고, 해당 책 정보를 출력하는 코드를 작성하라. 단, 세 권의 책이 출판된 연도는 서로 다르다고 가정한다.



```
if d_list[0]['Year']==1950:  
    print(d_list[0])  
elif d_list[1]['Year']==1950:  
    print(d_list[1])  
elif d_list[2]['Year']==1950:  
    print(d_list[2])
```

- 4) 위 문제에서 사용된 변수 `d_list`를 = 기호를 사용하여 `d_list2`에 넣으라, `d_list[1]`의 연도정보를 바꾼 후, `d_list`를 출력하여 값을 확인하라.



```
d_list2 = d_list  
d_list2[1]['Year'] = 205  
print(d_list[1])
```

리스트

튜플

딕셔너리

집합



고급 데이터 타입 사용 실습 (3.7.5)

- 5) 하나의 의미에 대해 한국어, 영어, 일본어 단어를 그 정보로 포함하는 딕셔너리를 만들라.
단, key는 각각 'Korean', 'English', 'Japanese'로 하라.
영어, 일본어 대신 다른 언어로 대체해도 무방하다.



```
dict1 = {'Korean' : '사랑 ', 'English' : 'Love', 'Japanese': '愛'}
```

리스트

튜플

딕셔너리

집합



고급 데이터 타입 사용 실습 (3.7.5)

6) 위 문제에서와 같이 다국어 단어 정보를 가지는 딕셔너리 다수로 구성된 리스트를 만들라.



```
dict_list = [] #빈 리스트 생성
dict_tmp = {'Korean' : '사랑', 'English' : 'Love', 'Japanese': '愛'}
dict_list.append(dict_tmp)
dict_tmp = {'Korean' : '믿음', 'English' : 'Faith', 'Japanese': '信仰'}
dict_list.append(dict_tmp)
dict_tmp = {'Korean' : '소망', 'English' : 'Hope', 'Japanese': '希望'}
dict_list.append(dict_tmp)
dict_tmp = {'Korean' : '행복', 'English' : 'Happiness', 'Japanese': '幸せ'}
dict_list.append(dict_tmp)
print(dict_list)
```

리스트

튜플

딕셔너리

집합



목 차

3.6 조건문

3.7 고급 데이터 타입

3.8 반복문

3.9 Numpy 배열

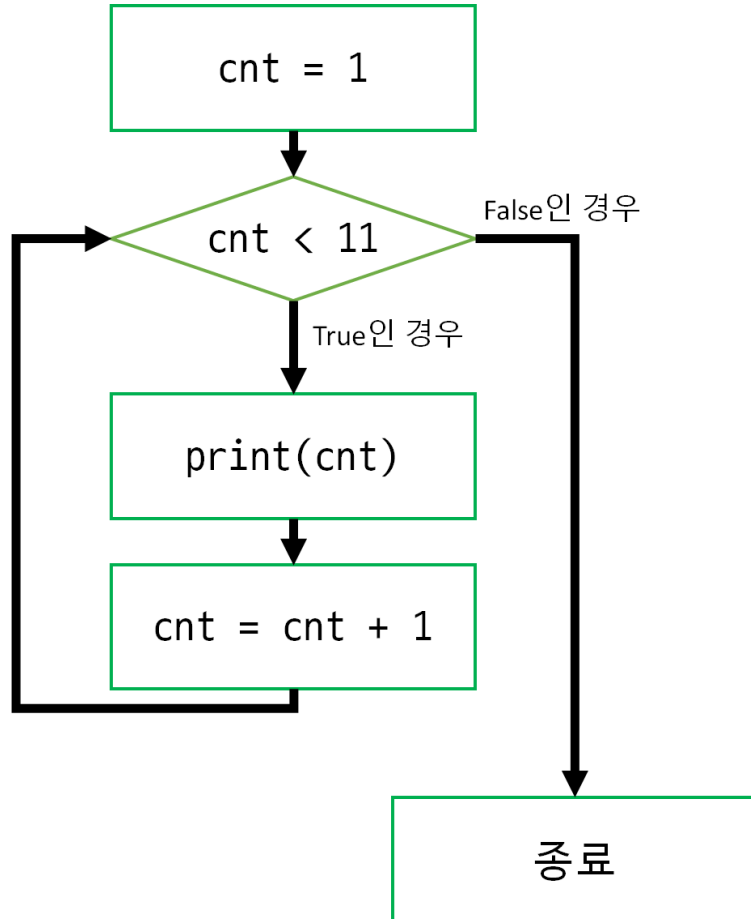
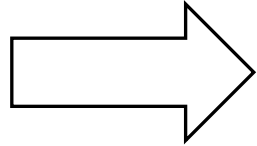
3.10 사용자 정의 함수



코드 실행의 순서

- 위에서 아래로
- = 오른쪽 먼저 계산 후 = 왼쪽으로 assign
- () 안에서 바깥쪽으로

```
print( 1 )  
print( 2 )  
print( 3 )  
print( 4 )  
print( 5 )  
print( 6 )  
print( 7 )  
print( 8 )  
print( 9 )  
print ( 10 )
```

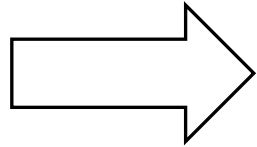




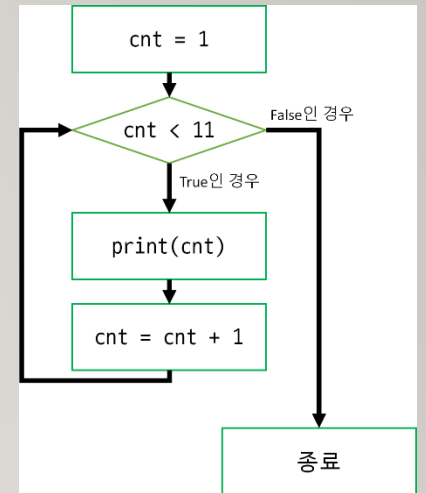
while 반복문

- while 과 조건식을 사용하여, 특정 조건을 만족하는 동안 반복

```
print( 1 )  
print( 2 )  
print( 3 )  
print( 4 )  
print( 5 )  
print( 6 )  
print( 7 )  
print( 8 )  
print( 9 )  
print ( 10 )
```



while 조건식:
 조건식이 참이면 수행할 코드 1
 조건식이 참이면 수행할 코드 2
 ...

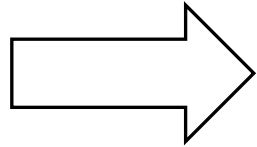




while 반복문

- while 과 조건식을 사용하여, 특정 조건을 만족하는 동안 반복

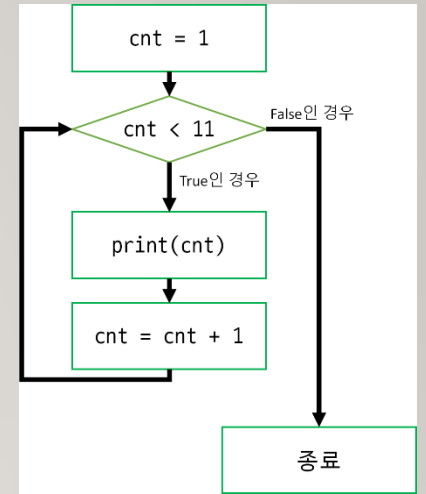
```
print( 1 )  
print( 2 )  
print( 3 )  
print( 4 )  
print( 5 )  
print( 6 )  
print( 7 )  
print( 8 )  
print( 9 )  
print ( 10 )
```



```
cnt = 1          # cnt라는 변수를 만들고 그 값을 1로 설정한다  
while cnt < 11:  # cnt가 11보다 작다면 아래의 코드를 실행한다  
    print(cnt)   # cnt의 값을 화면에 표시한다  
    cnt = cnt + 1 # cnt의 값에 1을 더한 후, 다시 cnt에 기록한다.
```

주석: 코드 설명문 표시

들여쓰기

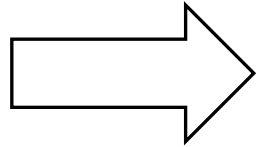




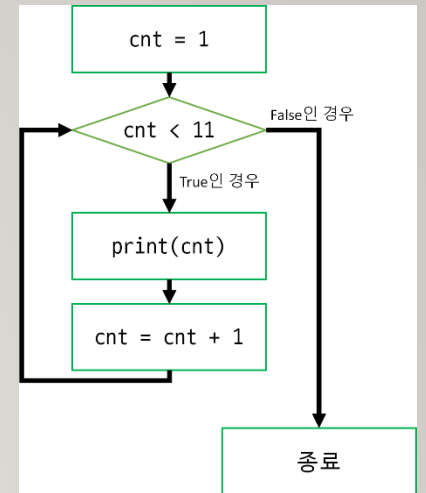
for 반복문

- for와 리스트를 사용하여, 리스트 내 값의 개수만큼 반복

```
print( 1 )  
print( 2 )  
print( 3 )  
print( 4 )  
print( 5 )  
print( 6 )  
print( 7 )  
print( 8 )  
print( 9 )  
print ( 10 )
```



```
for 변수 in 리스트:  
    수행할 코드1  
    수행할 코드2
```

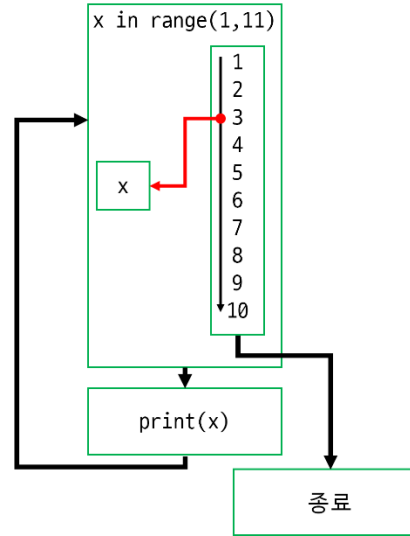
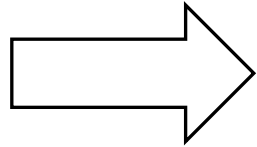




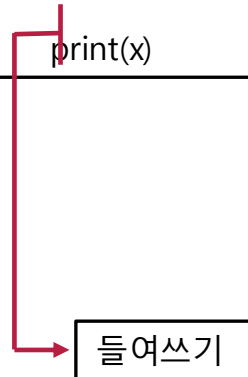
for 반복문

- for와 리스트를 사용하여, 리스트 내 값의 개수만큼 반복

```
print( 1 )  
print( 2 )  
print( 3 )  
print( 4 )  
print( 5 )  
print( 6 )  
print( 7 )  
print( 8 )  
print( 9 )  
print ( 10 )
```



```
for x in range(1,11): # range(1,11)은 1부터 10까지의 모든 자연수를 의미한다.  
    print(x)          # 따라서, x에는 각각의 수가 차례대로 대입된다  
                      # x를 출력한다
```



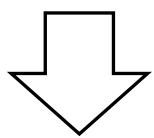


08. 반복문

반복문과 조건문의 활용

• 우박수 계산

```
n = 5          # 변수 n 의 값을 5로 설정한다.
n = n * 3 + 1  # n에 3을 곱하고 1을 더한 결과를 다시 n에 기록한다
n = n / 2      # n의 값을 가져와 2로 나누고 n에 다시 저장한다
n = n / 2      # n의 값을 가져와 2로 나누고 n에 다시 저장한다
n = n / 2      # n의 값을 가져와 2로 나누고 n에 다시 저장한다
n = n / 2      # n의 값을 가져와 2로 나누고 n에 다시 저장한다
print(n)       # n의 값을 화면에 표시한다.
```



홀수인 경우 3을 곱해서 1을 더함
 짝수인 경우 2로 나눔
 1이 될 때까지 반복

코드 (명령)	의미
n = 5	
n = n * 3 + 1	✓ $n * 3 + 1$ 계산 $\Rightarrow 16$ ✓ 계산된 결과를 n에 기록
n = n / 2	✓ $n / 2$ 계산 $\Rightarrow 8$ ✓ 계산된 결과를 n에 기록
n = n / 2	
n = n / 2	
n = n / 2	
print(n)	✓ n의 값 출력 (화면표시)



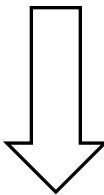
08. 반복문

반복문과 조건문의 활용

- 우박수 계산

```

▶ n = 5          # 변수 n의 값을 5로 설정한다.
  n = n * 3 + 1   # n에 3을 곱하고 1을 더한 결과를 다시 n에 기록한다
  n = n / 2       # n의 값을 가져와 2로 나누고 n에 다시 저장한다
  n = n / 2       # n의 값을 가져와 2로 나누고 n에 다시 저장한다
  n = n / 2       # n의 값을 가져와 2로 나누고 n에 다시 저장한다
  n = n / 2       # n의 값을 가져와 2로 나누고 n에 다시 저장한다
  print(n)        # n의 값을 화면에 표시한다.
    
```



- 원리를 찾아 반복문으로 변환
홀수인 경우 3을 곱해서 1을 더함
짝수인 경우 2로 나눔
1이 될 때까지 반복

```

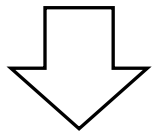
▶ n = 17          # n = 17로 설정
  print(n)        # n의 값을 화면에 표시
  while n != 1:   # n == 1이 되면 반복문을 빠져나간다. (n != 1 인 동안 계속 반복)
    if n % 2 == 0: # n이 짝수라면
      n = n / 2    # 2로 나누어 저장한다
    else:         # n이 홀수라면
      n = n * 3 + 1 # 3을 곱하고 1을 더하여 저장한다
    print(n)      # 변경된 값을 표시한다.
  print('콜라츠 추측이 성립함이 확인되었습니다')
    
```

코드 (명령)	의미
n = 5	
n = n * 3 + 1	✓ n * 3 + 1 계산 => 16 ✓ 계산된 결과를 n에 기록
n = n / 2	✓ n / 2 계산 => 8 ✓ 계산된 결과를 n에 기록
n = n / 2	
n = n / 2	
n = n / 2	
print(n)	✓ n의 값 출력 (화면표시)



break와 continue

- break
반복문을 빠져나간다
- continue
이후 코드를 실행하지 않고, 반복문의 맨 처음으로 이동한다.
- break문을 사용해 누적합이 10보다 커지면 멈추는 코드

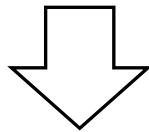


변수 sum의 값을 0으로 지정한 후,
1부터 10까지의 숫자를 계속 sum에 더하다가,
sum의 값이 10보다 커지면 종료



```
sum = 0
for i in range(1,11):
    sum = sum + i
    if(sum > 10):
        break;
print(sum)
```

- continue 문을 사용해 1부터 10까지의 짝수들
모두 더하는 코드



변수 sum의 값을 0으로 지정한 후,
1부터 10까지의 숫자를 계속 sum에 더하지만,
짝수인 경우 더하지 않고 반복문의 처음으로 이동



```
sum = 0
for i in range(1,11):
    if(i%2 == 0):
        continue
    sum = sum + i
print(sum)
```




반복문 사용 실습 (3.8.4)

1) 리스트 변수 `v`에 다음과 값이 저장되어 있을 때,

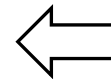
```
v = [3,4,5,6]
```

리스트에 포함된 값들의 평균을 계산하라.

반복문을 사용하여 작성한 코드



```
sum = 0
for x in v:
    sum = sum + x
avg = sum / len(v)
```



반복문 없이 작성한 코드



```
avg = ( v[0] + v[1] + v[2] + v[3] ) / 4
```



반복문 사용 실습 (3.8.4)

2) 10,000 이하의 2의 거듭제곱수 중 가장 큰 수를 찾는 코드를 작성하라.

while 반복문을 사용하여 작성한 코드



```
n = 0
power_of_n = 1
while power_of_n <= 10000:
    n = n + 1
    power_of_n = power_of_n * 2
print(f'The answer is {n-1}')
```

for 반복문과 break를 사용하여 작성한 코드



```
for n in range(10000):
    power_of_n = 2**n
    if(power_of_n > 10000):
        break
print(f'The answer is {n-1}')
```



반복문 사용 실습 (3.8.4)

3) 책 세 권의 정보가 다음과 같이 리스트형 변수인 book_list에 저장되어 있다고 하자.

```
book_list = [{'Title' : 'The Chronicles of Narnia',  
             'Year' : 1950,  
             'Author' : 'C. S. Lewis',  
             'Country' : 'UK',  
             'Genre' : 'Fantasy'},  
             {'Title' : 'Peter Pan',  
             'Year' : 1911,  
             'Author' : 'J. M. Barrie',  
             'Country' : 'UK',  
             'Genre' : 'Fantasy'},  
             {'Title' : 'The education of little tree',  
             'Year' : 1976,  
             'Author' : 'Forrest Carter',  
             'Country' : 'US',  
             'Genre' : 'Historical Fiction'}]
```

책 세 권 중 1976년에 출판된 책을 찾고, 해당 책 정보를 출력하는 코드를 작성하라.
단, 세 권의 책이 출판된 연도는 서로 다르다고 가정한다.

반복문을 사용하여 작성한 코드



```
for book in book_list:  
    if book['Year']==1976:  
        print(book)
```

반복문 없이 작성한 코드



```
if book_list[0]['Year']== 1976 :  
    print(book_list[0])  
elif book_list[1]['Year']== 1976 :  
    print(book_list[1])  
elif book_list[2]['Year']== 1976 :  
    print(book_list[2])
```





반복문 사용 실습 (3.8.4)

4) 다음과 같이 다국어 단어 정보를 가지는 딕셔너리 리스트가 있을 때,

```
dict_list = [{'Korean' : '사랑', 'English' : 'Love', 'Japanese': '愛'},  
             {'Korean' : '믿음', 'English' : 'Faith', 'Japanese': '信仰'},  
             {'Korean' : '소망', 'English' : 'Hope', 'Japanese': '希望'},  
             {'Korean' : '행복', 'English' : 'Happiness', 'Japanese': '幸せ'}]
```

한국어 단어 '행복'에 해당하는 영어 단어를 찾아 출력하라.



```
for dict in dict_list:  
    if dict['Korean']=='행복':  
        print(dict['English'])
```



반복문 사용 실습 (3.8.4)

5) 100 이하의 자연수 중 3으로 나누어 떨어지지 않는 수를 모두 출력하는 코드를 작성하라.



```
for i in range(1,101):  
    if i % 3 != 0:  
        print(i)
```

6) 100과 200 사이의 자연수 중, 제곱근이 자연수인 수를 모두 출력하는 코드를 작성하시오



```
import numpy as np  
for i in range(100,201):  
    sq = np.sqrt(i)  
    if sq == int(sq):  
        print(f'{i} = {int(sq)} * {int(sq)}')
```



목 차

3.6 조건문

3.7 고급 데이터 타입

3.8 반복문

3.9 Numpy 배열

3.10 사용자 정의 함수



배열

- 1차원, 2차원, 고차원 상에 연속적으로 데이터를 둔 것

3	4	1	2	8	2	4	9
---	---	---	---	---	---	---	---

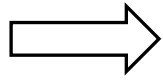
3	4	1	2
8	2	4	9



numpy 배열

- 파이썬 기본 라이브러리에서는 리스트를 통해 배열을 사용 가능
- 리스트는 복잡한 배열연산에 적합하지 않음
- 파이썬으로 구현된 딥러닝 코드는 numpy 배열을 사용
- Numpy의 array함수를 통해 리스트를 배열로 변환 가능

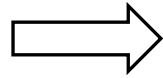
3	4	1	2	8	2	4	9
---	---	---	---	---	---	---	---



```
import numpy as np  
n = np.array([3,4,1,2,8,2,4,9])
```

- 2차원 리스트를 2차원 배열로 변환 가능

3	4	1	2
8	2	4	9



```
n = np.array([ [3,4,1,2], [8,2,4,9] ] )
```



배열의 생성

- array 함수, zeros, ones 함수를 사용하여 배열 생성 가능



```
n = np.array([ [3,4,1,2],[8,2,4,9] ] )  
print(n)
```

- zeros 함수는 0으로 된 배열을, ones 함수는 값이 1로 된 배열을 생성



```
n = np.zeros( (3,5) )  
print(n)
```



```
m = np.ones( (3,5,2) )  
print(m)
```

- random.random 함수는 임의의 값을 가지는 배열을 생성



```
n = np.random.rand(3,5)  
print(n)
```



배열의 인덱싱

- 배열의 특정 위치를 선택하는 것
- 대괄호를 사용하여 배열 내의 좌표를 지정



```
n = np.array([ [3,4,1,2],[8,2,4,9] ] )  
print(n[1,2])
```



```
n[1,2] = 0  
print(n[1,2])
```

- Boolean 배열을 사용하여 배열의 값 일부를 인덱싱 가능
- Boolean 배열과 원본 배열의 크기는 동일해야 함



```
n = np.array([ [3,4,1,2],[8,2,4,9], [2,1,8,7] ] )  
b = np.array([ [True,False,True,False],[True,False,True,False],[True,False,True,False] ] )  
n[b] = 0  
print(n)
```

- 조건문을 사용하여 해당 조건을 만족하는 배열 위치를 인덱싱 가능
- 조건문의 결과가 Boolean 배열로 나오기 때문



```
import numpy as np  
n = np.array([[3,4,1,2], [8,2,4,9], [2,1,8,7]])  
n[n<5] = 0  
print(n)
```




배열의 모양 변경

- 배열.shape 코드를 통해 배열의 모양(차원별 크기) 확인 가능

```
▶ print(n)  
print(n.shape)
```

- expand_dims 함수를 사용하여 차원을 추가
- 추가된 차원의 크기는 1

```
▶ n = np.expand_dims(n,1) #1번 차원을 추가  
print(n)  
print(n.shape)
```



```
n = np.array([ [3,4,1,2],[8,2,4,9] ] )
```



```
n = np.expand_dims(n,-1) #마지막 차원을 추가  
print(n)  
print(n.shape)
```

- Reshape 함수를 사용하여 배열의 모양을 변경
- 변경할 모양의 크기는 튜플 형태로 지정
- 배열의 전체 크기는 변경 전/후로 동일해야 함 (e.g. 3x6 → 9x2 가능. 3x6 → 4x5 불가능)

```
▶ m = np.reshape(n,(2,4))  
print(n)  
print(m)
```



슬라이싱

- 배열의 특정 범위를 선택. 인덱싱의 한 종류. step 이 생략되면 1로 간주됨

변수이름[start:end:step]

- 0,1,2 위치 선택

 `n[0:3]`

- 1,3,5 위치의 값을 0으로 변경

 `n[1:6:2] = 0`

- start 값을 생략하면 0으로 간주.

 `n[:3]`

- 2,3,다차원에서도 적용 가능

 `import numpy as np
n = np.array([[3,4,1,2], [8,2,4,9], [2,1,8,7]])
m = n[1,:2]
m[0,:] = 0
print(n)`



`n = np.array([ 3,4,1,2, 8,2,4,9] )`

- 1,3,5 위치 선택

 `n[1:6:2]`

- 1,3,5 위치의 값을 각각 -1, -2, -3으로 변경

 `n[1:6:2] = [-1,-2,-3]`

- end 값을 생략하면 배열의 끝까지 선택됨

 `n[3:]`

- 선택된 범위에 값을 넣는 경우, 선택된 인덱스와 값의 개수가 동일해야 함

 `n[1:6:2] = [-1,-2]` ❌





numpy 라이브러리의 기타 함수들

- max: 최댓값
- min: 최솟값
- mean: 평균
- std: 표준편차
- sum: 합계



```
max_v = np.max(data)
min_v = np.min(data)
mean_v = np.mean(data)
std_v = np.std(data)
sum_v = np.sum(data)

print(f'최댓값: {max_v}')
print(f'최솟값: {min_v}')
print(f'평균: {mean_v}')
print(f'표준편차: {std_v}')
print(f'합계: {sum_v}')
```

변수 뒤에 점을 찍는 방식으로도 사용 가능



```
max_v = data.max()
min_v = data.min()
mean_v = data.mean()
std_v = data.std()
sum_v = data.sum()
```




numpy 라이브러리의 기타 함수들

- 통계를 각 축(차원)에 대해 계산 가능



```
n = np.array([ 3,4,1,2, 8,2,4,9] )
```



```
max_v = np.max(data, axis=0)
min_v = np.min(data, axis=0)
mean_v = np.mean(data, axis=0)
std_v = np.std(data, axis=0)
sum_v = np.sum(data, axis=0)
```

```
print(f'최댓값: {max_v}')
print(f'최솟값: {min_v}')
print(f'평균: {mean_v}')
print(f'표준편차: {std_v}')
print(f'합계: {sum_v}')
```



numpy 라이브러리의 기타 함수들

- 행/열 별 통계 실습

	50m달리기 (초)	멀리뛰기 (초)	오래매달리기(초)	오래달리기(초)	윗몸일으키기 (횟수)
0번 축	8.5	200	60.2	450.3	50
	12.3	230.5	56.3	390.2	35
	9.2	190	30	500.4	60
	1번 축				



```
data = np.array([[8.5,      200,      60.2, 450.3,  50],
                 [12.3,    230.5,    56.3, 390.2,   35],
                 [9.2,     190,      30,   500.4,  60]])
```

```
mean_v = np.mean(data, axis=0)
std_v = np.std(data, axis=0)
print(f'50m달리기: {mean_v[0]:.2f}±{std_v[0]:.2f}')
print(f'멀리뛰기: {mean_v[1]:.2f}±{std_v[1]:.2f}')
print(f'오래 매달리기: {mean_v[2]:.2f}±{std_v[2]:.2f}')
print(f'오래달리기: {mean_v[3]:.2f}±{std_v[3]:.2f}')
print(f'윗몸일으키기: {mean_v[4]:.2f}±{std_v[4]:.2f}')
```



numpy 라이브러리의 기타 함수들

- argmax: 최댓값의 인덱스
- argmin: 최솟값의 인덱스



```
data = np.array([1,2,5,0,1])
idx_max = np.argmax(data)
idx_min = np.argmin(data)
print(f'최솟값은 {data[idx_min]} 입니다.')
print(f'최댓값은 {data[idx_max]} 입니다.')
```




numpy 라이브러리의 기타 함수들

- argmax: 최댓값의 인덱스
- argmin: 최솟값의 인덱스

	50m달리기 (초)	멀리뛰기 (초)	오래매달리기(초)	오래달리기(초)	윗몸일으키기 (횟수)
0번 측	8.5	200	60.2	450.3	50
	12.3	230.5	56.3	390.2	35
	9.2	190	30	500.4	60
	1번 측				



```
data = np.array([[8.5,      200,      60.2, 450.3,   50],
                 [12.3,    230.5,    56.3, 390.2,   35],
                 [9.2,     190,      30,   500.4,  60]])
```



```
idx_min = np.argmin(data, axis=0)
best_50m_idx = idx_min[0]
long_jump_of_best_50m_person = data[best_50m_idx, 1]
print(f'50m에서 가장 빠른 기록을 가지고 있는 학생은 {best_50m_idx}번 입니다.')
print(f'해당 학생의 멀리뛰기 기록은 {long_jump_of_best_50m_person}입니다')
```

- 슬라이싱 활용



```
data[data[:,1]>190, 4]  #멀리뛰기 기록이 190 초과인 학생들의 윗몸일으키기 기록 선택
```



목 차

3.6 조건문

3.7 고급 데이터 타입

3.8 반복문

3.9 Numpy 배열

3.10 사용자 정의 함수



10. 사용자 정의 함수

사용자 정의 함수의 정의

- 필요한 함수를 직접 만들어서 사용 가능
- def 키워드를 사용하여 정의
- 파라미터 이름 정의
- 반환값을 사용하는 경우 return 명령 사용

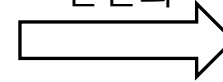
```
def 함수이름(파라미터1, 파라미터2, ...):
    수행할 코드1
    수행할 코드2
    return 반환값
```

- 파라미터로 받은 두 개의 값을 더하여 반환하는 함수



```
def add(v1, v2):
    sum = v1 + v2
    return sum
```

간결화



```
def add(v1, v2):
    return v1 + v2
```

- 정의한 함수의 활용



```
v = add(3,6)
print(v)
```




사용자정의 함수에서 기본값 지정

- 파라미터 이름에 대입 연산자를 사용하여 기본값 지정 가능
- 파라미터로 받은 두 개의 값을 더하여 반환하는 함수. 두 번째 파라미터는 기본값이 5로 지정됨



```
def add(v1, v2=5):  
    return v1 + v2
```

- 정의한 함수의 활용. 두 번째 파라미터는 생략 가능



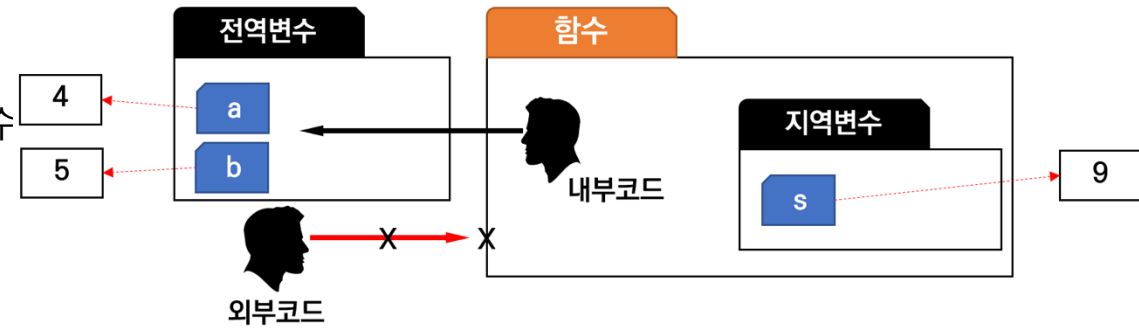
```
v = add(3)  
print(v)
```



10. 사용자 정의 함수

변수의 범위와 함수

- 함수 내부에서 만들어진 변수는 함수 밖에서 사용할 수 없음 → 지역 변수
- 함수 외부에서 만들어진 변수는 함수 안에서 사용 가능 → 전역 변수
- 전역 변수의 사용은 권장되지 않으며, 꼭 필요한 경우에 한해서 사용



- 변수 a, b는 함수 외부에서 값이 지정(생성)됨



```
def add():
    s = a + b
    return s
```



```
a = 4
b = 5
c = add()
print(c)
```



```
def add(v1, v2):
    s = v1 + v2
    return s
```



```
a = 4
b = 5
c = add(a, b)
print(s)
```

에러 발생



10. 사용자 정의 함수

사용자 정의함수의 활용

1) 책 정보를 가지고 있는 딕셔너리 리스트의 내용을 출력하는 함수



```
def printBook(book):  
    print("-----")  
    print(f"Title: {book['Title']}")  
    print(f"Published Year: {book['Year']}")  
    print(f"Country: {book['Country']}")  
    print(f"Genre: {book['Genre']}")
```



```
printBook(book_list[0])  
printBook(book_list[1])  
printBook(book_list[2])
```



```
book_list = [{'Title' : 'The Chronicles of Narnia',  
              'Year' : 1950,  
              'Author' : 'C. S. Lewis',  
              'Country' : 'UK',  
              'Genre' : 'Fantasy'},  
             {'Title' : 'Peter Pan',  
              'Year' : 1911,  
              'Author' : 'J. M. Barrie',  
              'Country' : 'UK',  
              'Genre' : 'Fantasy'},  
             {'Title' : 'The education of little tree',  
              'Year' : 1976,  
              'Author' : 'Forrest Carter',  
              'Country' : 'US',  
              'Genre' : 'Historical Fiction'}]
```




사용자 정의함수의 활용

2) Numpy 배열에서 통계정보를 계산하는 함수

- 임의의 값을 가지는 2차원 배열 3개(d1,d2,d3) 생성



```
d1 = np.random.rand(5,4)
d2 = np.random.rand(3,8)
d3 = np.random.rand(4,7)
```

- 주어진 배열로부터 통계정보를 계산하는 함수 정의



```
import numpy as np
def cal_stat(data_arr):
    avg = np.mean(data_arr)
    std = np.std(data_arr)
    max_idx = np.argmax(data_arr,0)
    return avg, std, max_idx
```

- 정의된 함수를 사용하여 데이터로부터 통계치 계산



```
avg1, std1, max_idx1 = cal_stat(d1)
avg2, std2, max_idx2 = cal_stat(d2)
avg3, std3, max_idx3 = cal_stat(d3)
```



10. 사용자 정의 함수

객체참조에 의한 파라미터의 전달

- 파이썬에서 파라미터로 정보를 전달할 때에는 '객체'가 아니라 객체를 가리키는 '변수'가 전달됨
- 리스트, numpy 배열 등을 전달하는 경우 값이 바뀌어 돌아오게 됨

- 데이터 정의



```
d = np.random.rand(5,4)
```

- 함수 정의. 전달받은 배열(arr)에서 [0,0] 위치의 값을 999로 바꿈

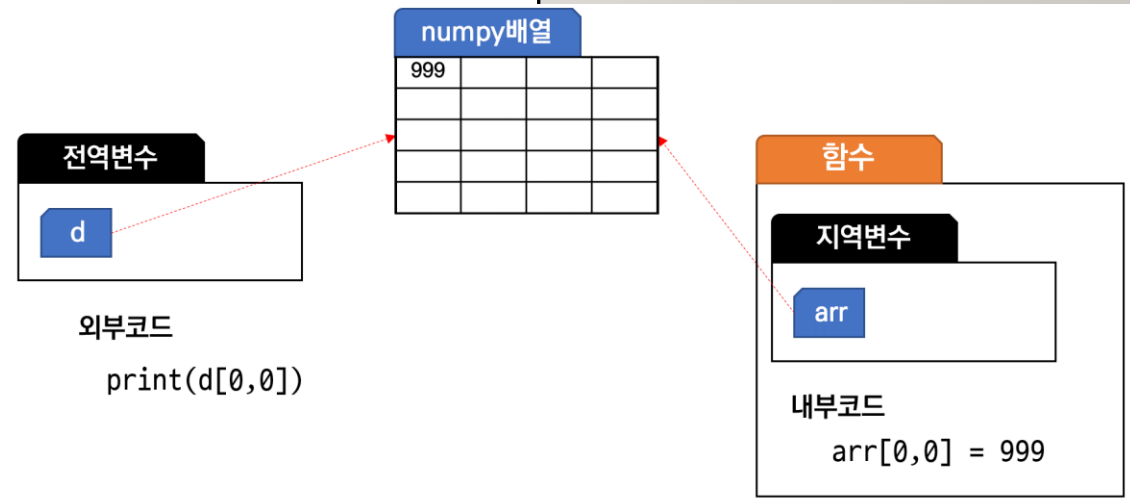


```
def change_var(arr):  
    arr[0,0] = 999
```

- 함수 호출 전/후의 값 비교



```
print(f'함수호출 전: {d[0,0]}')  
  
change_var(d)  
  
print(f'함수호출 후: {d[0,0]}')
```





10. 사용자 정의 함수

객체참조에 의한 파라미터의 전달

- (인덱싱을 사용하지 않고) 함수 내에서 변수 자체를 바꾸는 경우에는 함수 호출 전/후 값이 바뀌지 않음
- (인덱싱이 불가능한) 정수/실수 등의 변수는 함수 호출 전/후 값이 바뀌지 않음

- 데이터 정의



```
d = np.random.rand(5,4)
```

- 함수 정의. 새로운 변수(arr)에 999를 대입함

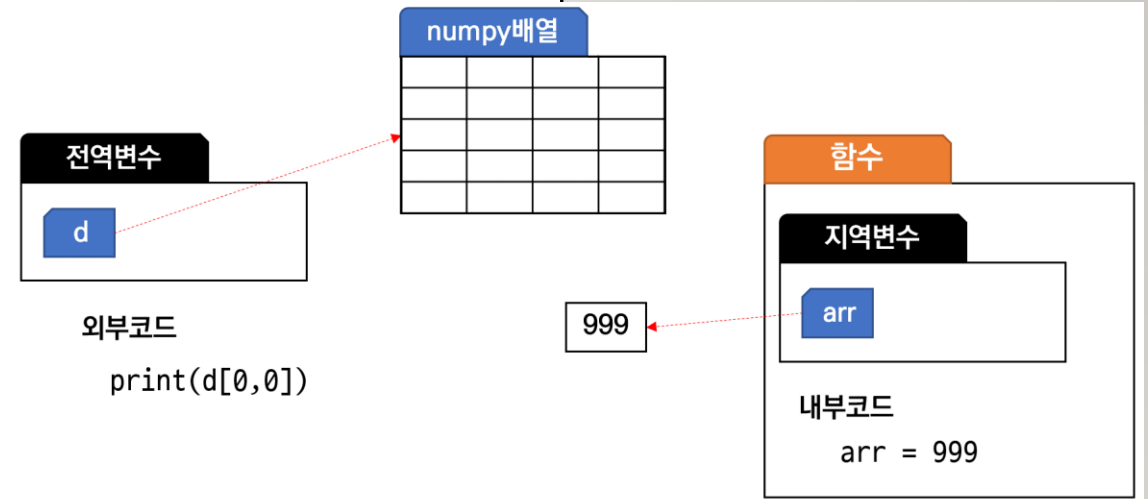


```
def change_var(arr):  
    arr = 999
```

- 함수 호출 전/후의 값 비교



```
print(f'함수호출 전: {d[0,0]}')  
  
change_var(d)  
  
print(f'함수호출 후: {d[0,0]}')
```



63

감사합니다

THANK YOU